



Chương 3. Kiểm thử Hộp trắng

La Quốc Thắng

1

Nội dung chính

1. Tổng quan về kiểm thử hộp trắng (1)
2. Kiểm thử đường dẫn cơ sở (2)
3. Kiểm thử điều kiện (3)
4. Kiểm thử vòng lặp (4)
5. Kiểm thử luồng điều khiển (5)

2

Tổng quan về kiểm thử hộp trắng

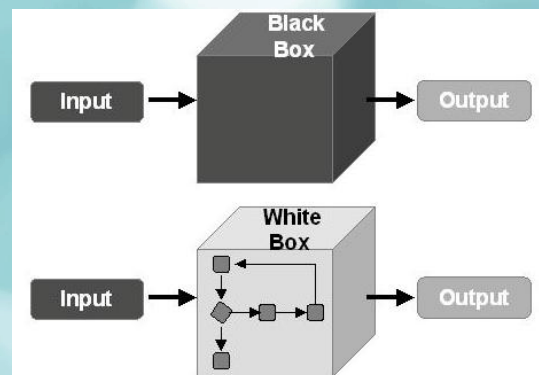
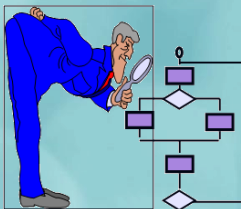
Chương 3. Kiểm thử Hộp trắng

3

3

Kiểm thử hộp trắng

- Đối tượng: Mã nguồn
- Cấp độ: Các module đơn vị
- Nội dung kiểm thử:
 - Chi tiết thủ tục (thuật toán)
 - Đường đi logic (luồng điều khiển)
 - Trạng thái của chương trình (dữ liệu)



Chương 3. Kiểm thử Hộp trắng

4

4

Kiểm thử hộp trắng

- Kiểm thử những gì:
 - Mọi lệnh trong chương trình (đầy đủ).
 - Mọi điều kiện logic có thể xảy ra (rẽ nhánh).
 - Mọi chu trình trong chương trình (lặp lại).
 - Mọi cấu trúc dữ liệu được sử dụng (dữ liệu).
 - Mọi tiến trình từ đầu đến kết thúc (từng luồng điều khiển).
- Câu hỏi đặt ra: Yêu cầu? Lý do? Các kỹ thuật dùng và nội dung của kỹ thuật đó? Chiến lược sử dụng?

5

Kiểm thử hộp trắng

- Tuy nhiên, số lượng con đường là rất lớn.

```
int i, j, k;
for (i=1; i<=1000; i++)
  for (j=1; j<=1000; j++)
    for (k=1; k<=1000; k++)
      doSomething(i,j,k);
```

Có 1 con đường với
 $1000^3 = 1$ tỉ lệnh gọi

```
if (c1) s11 else s12;
if (c2) s21 else s22;
if (c3) s31 else s32;
//...
if (c32) s321 else s322;
```

Có $2^{32} = 4$ tỉ con đường

6

Kiểm thử hợp trắng

- Vấn đề đặt ra:
 - Kiểm thử hợp trắng **không thể bao phủ** được toàn bộ các trường hợp.
 - Vì vậy, cần xác định số lượng trường hợp **kiểm thử tối thiểu** sao cho vẫn đảm bảo độ tin cậy của **kết quả là tối đa**.
- Câu hỏi: *Làm thế nào để xác định được số trường hợp kiểm thử tối thiểu có thể mang lại độ tin cậy tối đa?*



Phủ kiểm thử

- Phủ kiểm thử (Coverage) là **tỷ lệ giữa số lượng thành phần thực sự được kiểm thử và tổng số thành phần liên quan** sau khi đã thực hiện các trường hợp kiểm thử được chọn.
- Phủ kiểm thử càng cao thì độ tin cậy của kiểm thử càng lớn.
- Thành phần liên quan gồm:
 - Lệnh,
 - Điểm quyết định,
 - Điều kiện con,
 - Đường thi hành,
 - Hoặc tổ hợp của các thành phần trên.

Phủ cấp 0 & 1

- **Phủ cấp 0:** Chỉ kiểm thử những thành phần có thể kiểm thử được; phần còn lại để người dùng tự phát hiện và phản hồi. Đây là mức độ kiểm thử không thể hiện trách nhiệm đầy đủ.
- **Phủ cấp 1:** Kiểm thử bảo đảm mỗi lệnh trong chương trình được thực thi ít nhất một lần.

Phủ cấp 0 & 1

- Cho đoạn chương trình sau:

```
1 float foo(int a, int b, int c, int d) {
2     float e;
3     if (a==0)
4         return 0;
5     int x = 0;
6     if ((a==b) || ((c==d) && bug(a)))
7         x = 1;
8     e = 1/x;
9     return e;
10 }
```

- Với hàm foo bên cạnh, ta chỉ cần 2 trường hợp kiểm thử sau đây là đạt 100% phủ cấp 1:

1. foo(0,0,0,0), trả về 0
2. foo(1,1,1,1), trả về 1

- Tuy nhiên không phát hiện lỗi chia 0 ở dòng lệnh 8.

Phủ cấp 2

- Kiểm thử bảo đảm mỗi điểm quyết định được thực hiện ít nhất một lần cho cả hai trường hợp đúng (TRUE) và sai (FALSE).
- Mức kiểm thử này gọi là phủ các nhánh (branch coverage).
- Phủ các nhánh đồng thời bảo đảm phủ được các lệnh trong chương trình.

11

Phủ cấp 2

- Cho đoạn chương trình sau:

```
1 float foo(int a, int b, int c, int d) {  
2     float e;  
3     if (a==0)  
4         return 0;  
5     int x = 0;  
6     if ((a==b) || ((c==d) && bug(a)))  
7         x = 1;  
8     e = 1/x;  
9     return e;  
10 }
```

Line	Predicate	True	False
3	(a == 0)	Test Case 1 foo(0, 0, 0, 0) return 0	Test Case 2 foo(1, 1, 1, 1) return 1
6	((a==b) OR ((c == d) AND bug(a)))	Test Case 2 foo(1, 1, 1, 1) return 1	Test Case 3 foo(1, 2, 1, 2) return 1

- Với hai trường hợp kiểm thử 1 và 2. ta chỉ đạt $\frac{3}{4} = 75\%$ phủ các nhánh.
- Nếu bổ sung thêm trường hợp kiểm thử 3 thì mới đạt 100% phủ các nhánh.

12

Phủ cấp 2

- Cho đoạn chương trình sau:

```
1 float foo(int a, int b, int c, int d) {
2     float e;
3     if (a==0)
4         return 0;
5     int x = 0;
6     if ((a==b) || ((c==d) && bug(a)))
7         x = 1;
8     e = 1/x;
9     return e;
10 }
```

Dòng	Nhánh	TC1	TC2	Đã bao phủ
3	true	✓		✓
3	false		✓	✓
6	true		✓	✓
6	false	✗	✗	✗

- Với hai trường hợp kiểm thử 1 và 2. ta chỉ đạt $\frac{3}{4} = 75\%$ phủ các nhánh.
- Nếu bổ sung thêm trường hợp kiểm thử 3 thì mới đạt 100% phủ các nhánh.

Phủ cấp 3

- Kiểm thử bảo đảm mỗi điều kiện con (sub-condition) của từng điểm quyết định được thực hiện ít nhất một lần với cả hai trường hợp đúng (TRUE) và sai (FALSE).
- Mức kiểm thử này gọi là phủ các điều kiện con (sub-condition coverage).
- Phủ các điều kiện con chưa chắc đảm bảo phủ được các nhánh (branch coverage).

Phủ cấp 3

- Cho đoạn chương trình sau:

```

1 float foo(int a, int b, int c, int d) {
2     float e;
3     if (a==0)
4         return 0;
5     int x = 0;
6     if ((a==b) || ((c==d) && bug(a)))
7         x = 1;
8     e = 1/x;
9     return e;
10 }

```

Predicate	True	False
a ==0	Test Case 1 foo(0, 0, 0, 0) return 0	Test Case 2 foo(1, 1, 1, 1) return 1
(a==b)	Test Case 2 foo(1, 1, 1, 1) return value 0	Test Case 3 foo(1, 2, 1, 2) division by zero!
(c==d)		Test Case 3 foo(1, 2, 1, 2) division by zero!
bug(a)		

Chương 3. Kiểm thử Hộp trắng

15

15

Phủ cấp 4

- Kiểm thử bảo đảm mỗi điều kiện con (sub-condition) của từng điểm quyết định được thực hiện ít nhất một lần với cả hai trường hợp đúng (TRUE) và sai (FALSE), đồng thời điểm quyết định cũng phải được kiểm thử cho cả hai nhánh.
- Mức kiểm thử này gọi là phủ các nhánh và điều kiện con (branch & sub-condition coverage).

Chương 3. Kiểm thử Hộp trắng

16

16

Phủ kiểm thử

Mức phủ	Ý nghĩa chính	Mục tiêu kiểm thử
Phủ cấp 0	Kiểm thử cơ bản, chưa có trách nhiệm	Chạy cái gì dễ chạy, còn lại để người dùng phản ánh
Phủ cấp 1	Phủ các lệnh (statement coverage)	Mỗi lệnh được thực thi ít nhất một lần
Phủ cấp 2	Phủ các nhánh (branch coverage)	Mỗi điểm quyết định được kiểm thử cho cả TRUE và FALSE
Phủ cấp 3	Phủ các điều kiện con (sub-condition coverage)	Mỗi điều kiện con được kiểm thử cho TRUE và FALSE
Phủ cấp 4	Phủ toàn diện: nhánh & điều kiện con	Cả nhánh lẫn điều kiện con đều được kiểm thử đầy đủ

Yêu cầu kiểm thử hộp trắng

- **Mỗi con đường độc lập** trong một module cần được thực hiện ít nhất một lần.
- **Mỗi ràng buộc logic** phải được kiểm thử ở cả hai phía: đúng (TRUE) và sai (FALSE).
- Tất cả các **vòng lặp tại biên** và tại các **biên vận hành** đều phải được thực hiện.
- **Mỗi cấu trúc dữ liệu** nội tại được sử dụng cần được kiểm thử để bảo đảm hiệu lực thực thi của nó.

“Phủ luồng logic toàn diện” (full path-oriented coverage)

Vì sao cần kiểm thử hộp trắng?

- Các logic sai và giả thiết không đúng có mối quan hệ tỷ lệ nghịch với xác suất một con đường logic được thi hành.
- Thực tế, mọi con đường logic đều có thể được thi hành nếu có cơ sở hoặc điều kiện thích hợp.
- Có những lỗi ngẫu nhiên xuất hiện trên các con đường logic mà ta không kiểm tra được.

Lưu ý

- Không thể bao phủ tất cả các trường hợp kiểm thử mong muốn chỉ với một kỹ thuật kiểm thử duy nhất.
- Kiểm thử phần mềm gồm nhiều khía cạnh khác nhau: logic nội tại, hành vi bên ngoài,...
- Mỗi kỹ thuật kiểm thử có ưu điểm riêng và không thể phát hiện toàn bộ lỗi tiềm ẩn.
- Trước khi áp dụng các kỹ thuật kiểm thử hộp trắng, cần đọc và phân tích mã nguồn.
- Việc đọc mã nguồn có vai trò quan trọng để xác định các đường logic, nhánh điều kiện, vòng lặp và các hàm chưa được gọi đến.

Kiểm thử đường dẫn cơ sở

22

Kiểm thử đường cơ sở (Basis path)

- Định nghĩa: Thu được các trường hợp thử nghiệm từ các đường cơ sở, được xác định dựa trên đồ thị dòng của chương trình.
- Các bước thực hiện:
 1. Xây dựng đồ thị dòng dựa trên logic chương trình.
 2. Tính toán độ phức tạp Cyclomatic của đồ thị dòng.
 3. Xác định các đường cơ sở.
 4. Kiểm tra xem số lượng đường cơ sở có vượt quá độ phức tạp Cyclomatic hay không.
 5. Thiết kế các trường hợp thử nghiệm để kiểm tra các đường cơ sở đó.

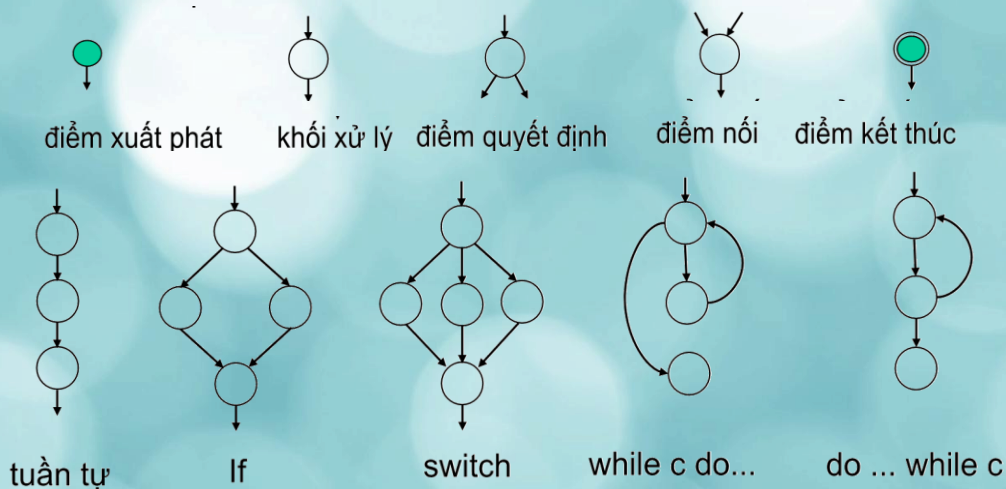
23

Đồ thị dòng (Flow graph)

- Đồ thị dòng là một kỹ thuật dựa trên cấu trúc điều khiển của chương trình, gần giống với đồ thị luồng điều khiển.
- Cách xây dựng từ đồ thị luồng điều khiển:
 1. Gộp các lệnh tuần tự lại.
 2. Thay các lệnh rẽ nhánh và điểm kết thúc của các đường điều khiển bằng một nút vị tự.
- Cấu trúc của đồ thị dòng:
 - Mỗi nút (dạng hình tròn) biểu diễn một hoặc một nhóm lệnh tuần tự, rẽ nhánh hoặc điểm hội tụ của các đường điều khiển.
 - Mỗi cạnh nối giữa hai nút biểu diễn dòng điều khiển trong chương trình.

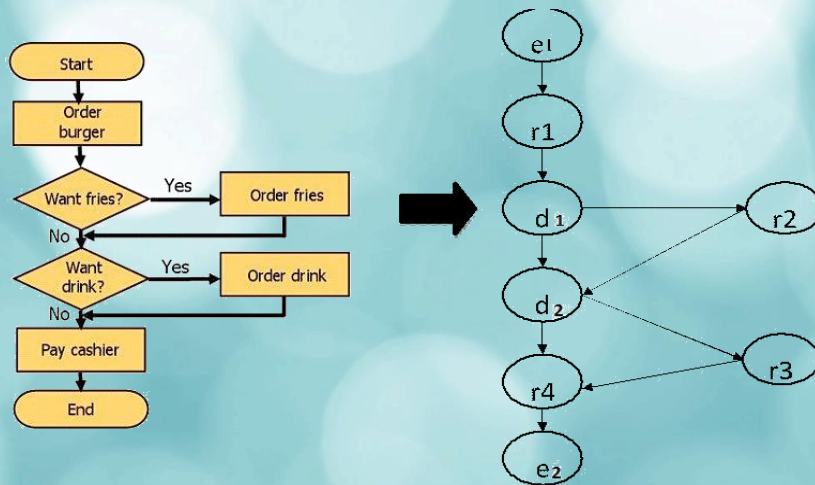
24

Đồ thị dòng



25

Đồ thị dòng



Đồ thị dòng (bên phải) có 8 nút, 9 cung.

26

Độ phức tạp của chu trình

- Để đảm bảo kiểm thử đầy đủ:
 - Cần xác định tất cả các đường điều khiển độc lập trong chương trình — tức là các đường đi khác biệt với các đường khác ít nhất một câu lệnh.
 - Số lượng đường điều khiển độc lập của một chương trình chính là giới hạn trên của số lượng trường hợp kiểm thử cần thực hiện.
 - Chỉ số này được gọi là độ phức tạp chu trình của chương trình.
 - Các đường điều khiển độc lập trong chương trình tương ứng với các đường điều khiển độc lập trong đồ thị dòng — việc xác định trong đồ thị giúp đơn giản hóa quá trình kiểm thử.

27

Độ phức tạp của chu trình

- Độ phức tạp McCabe (Cyclomatic complexity) theo công thức:

$$V(G) = E - N + 2P$$

$$V(G) = P + 1$$

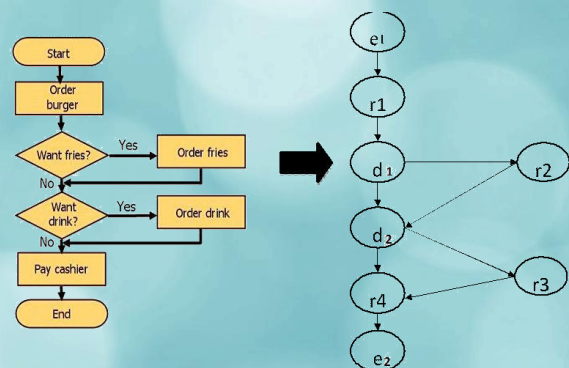
- Trong đó:
 - E: số cung (edge),
 - N: số nút (node),
 - P: số miền độc lập (thường là 1 nếu đồ thị liên thông), đối với công thức dưới thì P là số nút vị tử (nút chứa điều kiện).
- Độ phức tạp Cyclomatic chính là số đường thi hành tuyến tính độc lập cơ bản cần kiểm thử.

Độ phức tạp của chu trình

- Từ ví dụ đồ thị dòng bên cạnh, ta có:

$$V(G) = 9 - 8 + 2 = 3$$

➔ Số trường hợp kiểm thử tối thiểu để bao phủ toàn bộ các đường logic là 3.



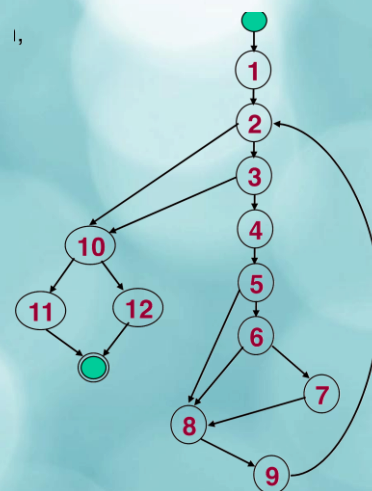
Quy trình xác định các đường tuyến tính độc lập (theo Tom McCabe)

1. Xác định đường cơ bản phổ biến nhất.
2. Để chọn đường thứ 2, thay đổi cung xuất phát của nút điều kiện đầu tiên và cố gắng giữ tối đa các đường còn lại.
3. Để chọn đường thứ 3, dùng đường cơ bản nhưng thay đổi cung xuất phát của nút điều kiện thứ hai và cố gắng giữ tối đa các đường còn lại.
4. Tiếp tục thay đổi cung xuất phát cho từng nút điều kiện tiếp theo trên đường cơ bản cho đến khi không còn nút điều kiện nào trong đường cơ bản nữa.
5. Lặp lại các đường cơ bản để xác định các đường mới xung quanh nó giống với bước 2, 3, 4 cho đến khi không tìm được đường tuyến tính độc lập nào nữa.

30

Ví dụ

```
double average(double value[], double min,
double max, int& tcnt, int& vcnt) {
double sum = 0;
int i = 1;
tcnt = vcnt = 0;
while (value[i] <= -999 && tcnt < 100) {
tcnt++;
if (min <= value[i] && value[i] <= max)
sum += value[i];
vcnt++;
}
i++;
}
if (vcnt > 0) return sum/vcnt;
return -999;
}
```



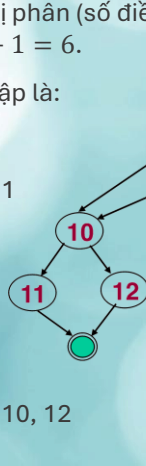
31

Ví dụ

Đồ thị bên có 5 nút quyết định nhị phân (số điều kiện) nên có độ phức tạp $C = 5 + 1 = 6$.

6 đường thi hành tuyến tính độc lập là:

- 1, 2, 10, 12
- 1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 10, 11
- 1, 2, 3, 4, 5, 8, 9, 2, 10, 12
- 1, 2, 3, 4, 5, 6, 8, 9, 2, 10, 12
- 1, (2, 3, 4, 5, 6, 7, 8, 9) × 100, 2, 3, 10, 11
- 1, (2, 3, 4, 5, 8, 9) × 100, 2, 3, 10, 12



```
double average(double value[], double min,
double max, int& tcnt, int& vcnt) {
double sum = 0;
int i = 1;
tcnt = vcnt = 0;
while (value[i] <= -999 && tcnt < 100) {
tcnt++;
if (min <= value[i] && value[i] <= max)
sum += value[i];
vcnt++;
}
i++;
if (vcnt > 0) return sum/vcnt;
return -999;
}
```

Chương 3. Kiểm thử Hộp trắng

32

32

Thiết kế trường hợp kiểm thử

- TC1: {1, 2, 10, 12}
 - Điều kiện:
 - value[1] == -999
 - vcnt == 0
 - Kết quả thực tế: -999

```
double average(double value[], double min,
double max, int& tcnt, int& vcnt) {
double sum = 0;
int i = 1;
tcnt = vcnt = 0;
while (value[i] <= -999 && tcnt < 100) {
tcnt++;
if (min <= value[i] && value[i] <= max)
sum += value[i];
vcnt++;
}
i++;
if (vcnt > 0) return sum/vcnt;
return -999;
}
```

Chương 3. Kiểm thử Hộp trắng

33

33

Thiết kế trường hợp kiểm thử

- TC2: {1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 10, 11}
- Điều kiện:
 1. $\text{value}[1] \neq -999$
 2. $\text{tcnt} < 100$
 3. $\text{min} \leq \text{value}[1] \leq \text{max}$
 4. $\text{value}[2] == -999$
 5. $\text{vcnt} > 0$
- Kết quả thực tế: sum/vcnt

```
double average(double value[], double min,
               double max, int& tcnt, int& vcnt) {
    double sum = 0;
    int i = 1;
    tcnt = vcnt = 0;
    while (value[i] <> -999 && tcnt < 100) {
        tcnt++;
        if (min <= value[i] && value[i] <= max)
            sum += value[i];
        vcnt++;
    }
    i++;
    if (vcnt > 0) return sum/vcnt;
    return -999;
}
```

Chương 3. Kiểm thử Hộp trắng

34

34

Thiết kế trường hợp kiểm thử

- TC3: {1, 2, 3, 4, 5, 8, 9, 2, 10, 12}
- Điều kiện:
 1. $\text{value}[1] \neq -999$
 2. $\text{tcnt} < 100$
 3. $\text{min} \leq \text{value}[1]$
 4. $\text{value}[1] > \text{max}$
 5. $\text{value}[2] == -999$
 6. $\text{vcnt} == 0$
- Kết quả thực tế: -999

```
double average(double value[], double min,
               double max, int& tcnt, int& vcnt) {
    double sum = 0;
    int i = 1;
    tcnt = vcnt = 0;
    while (value[i] <> -999 && tcnt < 100) {
        tcnt++;
        if (min <= value[i] && value[i] <= max)
            sum += value[i];
        vcnt++;
    }
    i++;
    if (vcnt > 0) return sum/vcnt;
    return -999;
}
```

Chương 3. Kiểm thử Hộp trắng

35

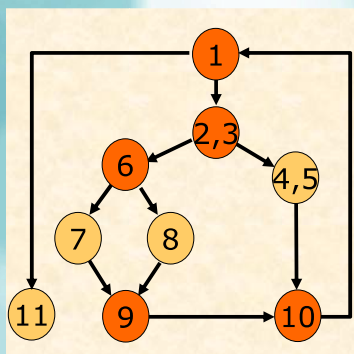
35

Ma trận kiểm thử

- Ma trận kiểm thử là một ma trận vuông có kích thước bằng số lượng các nút trong đồ thị dòng.
- Mỗi dòng và mỗi cột tương ứng với tên của một nút trong đồ thị.
- Mỗi ô trong ma trận biểu diễn tên của một cung nối từ nút ở dòng đến nút ở cột.
- Ma trận kiểm thử được sử dụng như một cấu trúc dữ liệu để kiểm tra các con đường cơ bản — cụ thể là số đường đi qua từng nút.

37

Ví dụ



	1	23	45	6	7	8	9	10	11
1		1							1
23			1	1					
45								1	
6					1	1			
7							1		
8							1		
9								1	
10	1								
11									

38

Ví dụ

	1	23	45	6	7	8	9	10	11
1		1							1
23			1	1					
45								1	
6					1	1			
7							1		
8							1		
9								1	
10	1								
11									

$2 - 1 = 1$

$2 - 1 = 1$

$1 - 1 = 0$

$2 - 1 = 1$

$1 - 1 = 0$

$1 - 1 = 0$

$1 - 1 = 0$

$V(G) = 3 + 1 = 4$

39

Kiểm thử điều kiện

40

Kiểm thử điều kiện

- Kiểm thử điều kiện là phương pháp thiết kế các trường hợp kiểm thử nhằm kiểm tra các điều kiện logic trong chương trình.
- Loại điều kiện:
 - Điều kiện đơn: Là một biến Boolean, có thể có toán tử phủ định hoặc là biểu thức quan hệ giữa hai biểu thức số học. Ví dụ: $C = (A \Theta B)$, trong đó A, B : biểu thức số học, Θ : phép so sánh như $<, \leq, =, >, \geq, \neq$.
 - Điều kiện phức hợp: Là tổ hợp của nhiều điều kiện đơn sử dụng các toán tử Boolean như NOT ($\neg$), OR ($\vee$), AND ($\wedge$). Ví dụ: $D = X_1 \wedge X_2 \wedge \dots \wedge X_n$, trong đó mỗi X_i là điều kiện đơn.
- Ghi chú: *Nếu điều kiện tổng thể sai thì ít nhất một trong các thành phần điều kiện đơn bên trong sẽ sai.*

Kiểu sai trong điều kiện logic

- **Sai biến Boolean:** Sử dụng sai tên biến hoặc giá trị không phù hợp kiểu Boolean.
- **Sai toán tử Boolean:** Dùng sai toán tử logic như AND, OR, NOT hoặc dùng không đúng ngữ nghĩa.
- **Sai số hạng trong biểu thức toán tử Boolean:** Thành phần (operand) trong biểu thức Boolean không hợp lệ hoặc không tương thích.
- **Sai toán tử quan hệ:** Dùng sai dấu so sánh như $<, <=, >, >=, =, !=$, gây ra kết quả không như mong đợi.
- **Sai biểu thức số học:** Cấu trúc hoặc phép toán trong biểu thức số học bị sai, ví dụ chia cho 0, phép toán không tương thích kiểu dữ liệu.

Chiến lược kiểm thử phân nhánh

- Kiểm thử từng điều kiện trong chương trình nhằm phát hiện lỗi không chỉ trong chính điều kiện đó mà còn ở các phần liên quan của chương trình.
- Nguyên tắc kiểm thử nhánh: *Với mỗi điều kiện phức hợp C , cần kiểm thử cả hai nhánh “đúng” (True, T) và “sai” (False, F). Đồng thời, mỗi điều kiện đơn trong C phải được kiểm thử ít nhất một lần.*

Chiến lược kiểm thử miền

- Chiến lược kiểm thử miền:
 - Với mỗi biểu thức quan hệ, cần thiết kế 3 hoặc 4 trường hợp kiểm thử đại diện cho các quan hệ: $<$, $>$, $=$ và có thể thêm $\neq$.
 - Khi biểu thức Boolean có số biến n nhỏ, việc thiết kế kiểm thử thuận lợi. Tuy nhiên, nếu n lớn thì việc kiểm thử trở nên khó khăn và phức tạp.
- Giải pháp kiểm thử nhảy cảm:
 - Áp dụng chiến lược kết hợp giữa kiểm thử nhánh (branch testing) và kiểm thử miền (relation testing) để bao phủ đầy đủ các tình huống quan trọng.
- Vấn đề đặt ra: *Làm thế nào để xác định đầy đủ tất cả các trường hợp cần kiểm thử?*

Chiến lược kiểm thử BRO

- BRO (**B**ranch and **R**elational **O**peration) là kỹ thuật kiểm thử nhánh kết hợp với toán tử quan hệ. Dừng ràng buộc điều kiện làm điều kiện cần thử để phát hiện sai sót ở nhánh và toán tử trong trường hợp chỉ xảy ra một lần và không có biến dừng chung.
- Giả sử: $D = X_1 \wedge X_2 \wedge \dots \wedge X_n$.
- Trong đó:
 - X_i là điều kiện đơn,
 - $\wedge$ là toán tử Boolean (AND).
- Yêu cầu: *Phải đặc tả đầu ra của từng điều kiện X_i sao cho tương ứng với điều kiện tổng thể D đã xác định — nhằm đảm bảo mỗi điều kiện đơn được kiểm thử độc lập nhưng vẫn phản ánh ảnh hưởng đến điều kiện tổng.*

Chiến lược kiểm thử BRO với điều kiện đầu vào

- Một ràng buộc X_i thuộc điều kiện D được gọi là được phủ bởi một sự thực thi của C nếu trong quá trình thực thi đó, đầu ra của mỗi điều kiện đơn X_i trong D thỏa mãn các ràng buộc tương ứng.
- Điều này có nghĩa: khi giá trị của điều kiện tổng D đã được xác định, ta cần tìm các ràng buộc mà mỗi X_i — là thành phần của D — phải thỏa mãn để bảo đảm giá trị của D là đúng.
- Với biến Boolean, ràng buộc đầu ra là giá trị True (T) hoặc False (F).
- Với biểu thức quan hệ $A \Theta B$, ràng buộc đầu ra chính là toán tử quan hệ Θ như $<$, $\leq$, $=$, $>$, $\geq$, $\neq$.

Chiến lược kiểm thử BRO

- Xét điều kiện $C = A \wedge B$, trong đó A và B đều là biến Boolean.
- Ràng buộc đầu ra:
 - Các cặp giá trị (T, T), (T, F), (F, T) biểu diễn kết quả của từng biến A và B .
- Chiến lược BRO yêu cầu các cặp đều được phủ bởi các thực thi của C :
 - Cặp (T, T) tương ứng với $C = T$.
 - Cặp (T, F) và (F, T) tương ứng với $C = F$.

Chiến lược kiểm thử BRO

- Xét điều kiện: $C = A \wedge (B = E)$, trong đó A là biến Boolean, $B = E$ là biểu thức quan hệ
- Các ràng buộc của C là các cặp giá trị đầu vào:
 - (T, T): A đúng, $B = E$ đúng
 - (T, F): A đúng, $B \neq E$ (có thể là $<$ hoặc $>$)
 - (F, T): A sai, $B = E$ đúng
- Chi tiết ràng buộc đầu ra:
 - (T, =): A đúng, $B = E$ đúng
 - (T, <): A đúng, $B < E \rightarrow B = E$ sai
 - (T, >): A đúng, $B > E \rightarrow B = E$ sai
 - (F, =): A sai, nhưng $B = E$ đúng

Kiểm thử vòng lặp

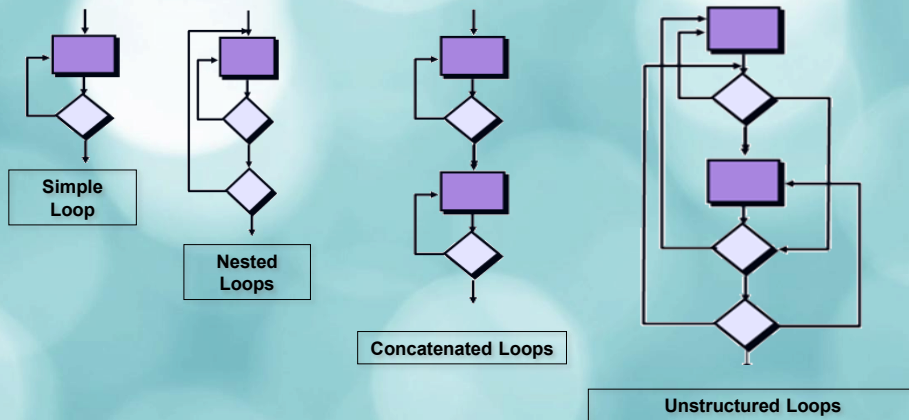
50

Kiểm thử vòng lặp (Loop testing)

- Định nghĩa: Kiểm thử vòng lặp tập trung vào việc đánh giá hiệu lực của các cấu trúc lặp trong chương trình, đặc biệt là các biến thể khác nhau của vòng lặp.
- Bốn kiểu vòng lặp:
 - **Simple loop (Vòng lặp đơn):** Vòng lặp có một cấu trúc duy nhất, không chứa vòng lặp khác bên trong.
 - **Nested loops (Vòng lặp lồng nhau):** Vòng lặp chứa bên trong nó một hoặc nhiều vòng lặp khác.
 - **Concatenated loops (Vòng lặp nối tiếp):** Nhiều vòng lặp thực hiện liên tiếp nhau, không lồng nhau.
 - **Unstructured loops (Vòng lặp phi cấu trúc):** Các vòng lặp được nối vòng, xen kẽ, lồng nhau phức tạp, không tuân theo một cấu trúc điều khiển rõ ràng.

51

Kiểm thử vòng lặp



Chương 3. Kiểm thử Hộp trắng

52

52

Kiểm thử vòng lặp đơn

- Kiểm thử vòng lặp với n là số vòng lặp tối đa cho phép — các trường hợp kiểm thử cần bao gồm:
 - **Bỏ qua vòng lặp:** Không thực hiện vòng lặp nào.
 - **Thực hiện một vòng lặp:** Lặp đúng một lần.
 - **Thực hiện hai vòng lặp:** Lặp hai lần liên tiếp.
 - **Thực hiện m vòng lặp, với $m < n$:** Đánh giá hành vi với số lần lặp trung gian.
- Thực hiện các biên quan trọng:
 - $n - 1$ lần.
 - n lần (số lặp tối đa dự kiến).
 - $n + 1$ lần (vượt giới hạn cho phép).

Chương 3. Kiểm thử Hộp trắng

53

53

Kiểm thử vòng lặp lồng nhau

- Các bước thực hiện:
 1. Bắt đầu từ vòng lặp trong cùng:
 - Đặt tất cả các vòng lặp bên ngoài ở giá trị tham số tối thiểu (số lần lặp thấp nhất).
 - Kiểm tra vòng lặp trong cùng với các giá trị: tối thiểu, tiêu biểu (trung bình hoặc giá trị thường gặp), tối đa (sát giới hạn cho phép).
 2. Di chuyển ra ngoài từng tầng vòng lặp:
 - Với mỗi vòng lặp bên ngoài:
 - Thiết lập nó theo ba mức kiểm thử như trên (min, trung bình, max)
 - Giữ các vòng lặp bên trong ở giá trị tiêu biểu ổn định
 - Tiếp tục lặp lại quy trình này cho tới khi đã kiểm thử vòng lặp ngoài cùng.

Kiểm thử vòng lặp ghép nối

- Chiến lược kiểm thử vòng lặp theo mối quan hệ giữa các vòng:
 - **Vòng lặp độc lập:** Khi mỗi vòng lặp không phụ thuộc vào vòng khác, ta kiểm thử như những vòng lặp đơn. Áp dụng các trường hợp kiểm thử tiêu chuẩn cho từng vòng riêng biệt.
 - **Vòng lặp nối tiếp có phụ thuộc:** Khi vòng lặp thứ nhất cung cấp giá trị đầu vào cho vòng lặp thứ hai, nghĩa là hai vòng có quan hệ liên kết, ta cần kiểm thử như vòng lặp lồng nhau. Điều này giúp đảm bảo bao phủ cả mối quan hệ đầu-vào giữa chúng.

Kiểm thử luồng điều khiển

Tiêu chuẩn bao phủ

- **Quy tắc dừng trong kiểm thử phần mềm:** Dùng để xác định mức độ đầy đủ của quá trình kiểm thử, nhằm biết khi nào có thể kết thúc việc kiểm thử một cách hợp lý.
- **Kiểm tra đo lường chất lượng:** Thực hiện đánh giá mức độ an toàn và hiệu quả liên quan đến từng tập kiểm thử để đảm bảo phần mềm đạt yêu cầu chất lượng.
- **Lựa chọn dữ liệu kiểm thử:** Một tập kiểm thử được xem là tương đương khi các trường hợp trong tập đáp ứng các tiêu chí tương tự — nghĩa là có khả năng phát hiện lỗi giống nhau và bao phủ hành vi hệ thống tương đương.

Tiêu chuẩn kiểm thử hộp trắng

- **Tiêu chuẩn luồng điều khiển (Control Flow Criteria):** Tập trung vào các đường đi và cấu trúc điều khiển của chương trình như: câu lệnh, nhánh, điều kiện, vòng lặp,...
- **Tiêu chuẩn luồng dữ liệu (Data Flow Criteria):** Tập trung vào hành vi của các biến và cách dữ liệu được định nghĩa, sử dụng, truyền đi trong chương trình.

Tiêu chuẩn luồng điều khiển

- Kiểm tra các biểu thức logic trong đó xác định các nhánh và cấu trúc vòng lặp của chương trình.
- Được sử dụng chủ yếu trong các ngành công nghiệp.
- Các tiêu chí phổ biến nhất bao gồm:
 1. Bao phủ câu lệnh (Statement Coverage)
 2. Bao phủ quyết định (Decision / Branch Coverage)
 3. Bao phủ điều kiện (Condition Coverage)
 4. Bao phủ quyết định/điều kiện (Decision/Condition Coverage)
 5. Bao phủ đa điều kiện (Multiple Condition Coverage)

Thứ tự tiêu chuẩn luồng điều khiển

- Một tiêu chuẩn kiểm thử A mạnh hơn tiêu chuẩn B nếu mọi tập kiểm thử thỏa mãn A cũng thỏa mãn B, nhưng không ngược lại. Điều này phản ánh rằng tiêu chuẩn A bao phủ logic chương trình sâu và rộng hơn tiêu chuẩn B.

Mức độ	Tiêu chuẩn kiểm thử	Đặc điểm nổi bật
1	Bao phủ câu lệnh (Statement coverage)	Kiểm tra mỗi câu lệnh được thực hiện ít nhất một lần
2	Bao phủ quyết định (Decision coverage)	Mỗi nhánh (true/false) của điều kiện được thực hiện
3	Bao phủ điều kiện (Condition coverage)	Mỗi điều kiện con được kiểm thử với cả hai giá trị T và F
4	Bao phủ quyết định/điều kiện (Decision/Condition coverage)	Kết hợp phủ nhánh và phủ điều kiện con
5	Bao phủ đa điều kiện (Multiple Condition coverage)	Bao phủ tất cả các tổ hợp có thể của điều kiện phức hợp

Bao phủ câu lệnh

- Code 1:

```
if ((A > 1) && (B == 0))
```

```
    X = X/A;
```

```
if ((A == 2) || (X > 1))
```

```
    X = X+1;
```

- Code 2:

```
if ((A > 1) || (B == 0))
```

```
    X = X/A;
```

```
if ((A == 2) || (X > 1))
```

```
    X = X+1;
```

- Định nghĩa: **Mỗi câu lệnh trong chương trình đều được thực hiện.**

- Ví dụ: Để thỏa mãn bao phủ câu lệnh đáp ứng đoạn Code 1, một trường hợp kiểm thử có thể là:

A = 2, B = 0, X = 3

- Như đã thấy, tất cả các câu lệnh đều được thực thi.

Bao phủ điều kiện

- Code 1:

```
if ((A > 1) && (B == 0))
```

```
    X = X/A;
```

```
if ((A == 2) || (X > 1))
```

```
    X = X+1;
```

- Định nghĩa: Mỗi câu lệnh trong chương trình được thực hiện ít nhất một lần và **mỗi điều kiện có thể xảy ra ít nhất một lần**.
- Một quyết định là một biểu thức Boolean bao gồm một hoặc nhiều điều kiện kết hợp bởi các từ nối hợp lí, chẳng hạn như “AND”, “OR” và “NOT”.

Bao phủ điều kiện

- Code 1:

```
if ((A > 1) && (B == 0))
```

```
    X = X/A;
```

```
if ((A == 2) || (X > 1))
```

```
    X = X+1;
```

- Code 2:

```
if ((A > 1) || (B == 0))
```

```
    X = X/A;
```

```
if ((A == 2) || (X > 1))
```

```
    X = X+1;
```

- Ví dụ: Để thỏa mãn bao phủ điều kiện đáp ứng đoạn Code 1, một trường hợp kiểm thử có thể là:

A = 2, B = 0, X = 2

A = 1, B = 0, X = 1

- Như đã thấy, tất cả các câu lệnh đều được thực thi.
- Nhưng điều gì sẽ xảy ra trong Code 2?

Kết luận

- Kiểm thử hộp trắng là kỹ thuật kiểm thử dựa trên cấu trúc bên trong của chương trình để kiểm tra logic và độ chính xác của mã nguồn.
- Bao gồm ba kỹ thuật chính là kiểm thử đường dẫn cơ sở, kiểm thử rẽ nhánh và kiểm thử vòng lặp.
- Tiêu chuẩn bao phủ có thể được sử dụng như quy tắc dừng và kiểm tra dữ liệu.
- Tiêu chuẩn luồng điều khiển được áp dụng trong các ngành công nghiệp, bao gồm bao phủ câu lệnh, bao phủ điều kiện, bao phủ quyết định/điều kiện và bao phủ đa điều kiện.

Tài liệu tham khảo

1. Bài giảng Kiểm thử phần mềm, Đại học Bách khoa Hà Nội
2. <https://www.frugaltesting.com/blog/understanding-bugs-defects-errors-faults-and-failures-in-software-testing>
3. <https://cs.gmu.edu/~offutt/softwaretest/>